NATURAL LANGUAGE PROCESSING

LABORATORY ASSIGNMENT – 06

Implementation of an NLP Case Study

Student Name	Shravani Balasaheb Kurkute
Roll No.	421129
PRN / Registration No.	22311926
Class / Division	BtechA
Academic Year	2026-2027
Faculty Name	Dr. Kapil N. Vhatkar

Assignment 6: Implementation of an NLP Case Study

Aim: To apply Natural Language Processing concepts to a real-world problem and develop, implement and evaluate an NLP-based case study using open-source tools and libraries.

1. Objectives

- Identify and formulate a real-world problem that can be addressed using NLP.
- Collect or select a suitable dataset and understand its characteristics.
- Apply appropriate text preprocessing and representation techniques.
- Design and implement an NLP-based solution using Python and open-source tools.
- Evaluate the developed system using suitable performance metrics.
- Analyse the results, limitations and possible future improvements.

2. Problem Statement

Select and implement any one suitable NLP case study. The case study should address a clearly defined real-world problem, use an appropriate dataset, apply relevant NLP techniques and provide measurable results.

3. Suggested Case Studies

- Spam Email / SMS Classification
- Fake News or Misinformation Text Classification
- Named Entity Recognition for a Domain
- Text Summarization
- Question Answering
- Document / News Classification
- Text Similarity or Plagiarism Detection
- Domain-specific Chatbot
- Information Retrieval
- Text-based Recommendation
- Any other NLP case study approved by the faculty

Selected Case Study: News Article Classification using TF-IDF and Multinomial Naive Bayes

4. Case Study Overview

Problem Domain: Text Classification using Natural Language Processing.

Application Area: Automatic classification and organization of news articles.

Target Users / Stakeholders: News organizations, journalists, researchers, and digital news readers.

Primary Objective: To develop an NLP-based system that automatically classifies news articles into Business, Entertainment, Politics, Sport, and Tech categories.

Brief description:

News Article Classification is a task in NLP where an automatic classification of the category is performed by the algorithm depending on the text of the news article. In our project, the BBC News dataset was used where the texts of the news articles are classified into one of five categories: Business, Entertainment, Politics, Sports, and Technology. For the transformation of the text into numerical features, TF-IDF was used. Then, the Multinomial Naive Bayes model is trained based on those features to classify the text into a category.

5. Problem Definition and Motivation

Various news websites have articles about many different topics. It will take time and efforts to manually classify every article into the respective class. An automated classifier could be used to classify articles based on their content.

This project aims to classify news articles into one of the five pre-defined classes using Natural Language Processing and machine learning techniques. TF-IDF algorithm represents the textual information contained in articles in numeric form, and Multinomial Naive Bayes algorithm finds the patterns for each class.

The algorithm predicts the class of an article and checks whether the prediction was right. This project shows how to use NLP to classify texts.

6. Objectives of the Proposed System

1. For analyzing and preprocessing of BBC News Dataset for Text Classification.
2. For cleaning the dataset by filling missing data, empty data points, and duplicates of news articles.
3. For converting the text of news articles into numbers using TF-IDF.
4. For training Multinomial Naive Bayes Classifier on labeled news articles.
5. For saving the trained classifier and testing it on unseen news articles.
6. For evaluating the classifier using accuracy, precision, recall, F1 score, confusion matrix, and classification report.

7. Dataset / Data Collection

Item	Details	Item	Details
Dataset Name	BBC Full Text and Category	Source / URL	https://www.kaggle.com/code/yufengdev/bbc-text-categorization/data
Language	English	Number of Records	2,225
Input Type	News article text	Output / Labels	Business, Entertainment, Politics, Sport, Tech
Training Set	1,700	Testing Set	426
Data Format	CSV	License / Usage	Non-commercial and research use; the original article content remains copyrighted by the BBC.

8. Dataset Analysis

The BBC News data set originally consisted of 2,225 rows with two variables namely, category and text. The variable text consists of the text content of the news articles, and the variable category consists of the label for the same.

There were five categories in the original data set: Business (510), Entertainment (386), Politics (417), Sport (511) and Tech (401). There were no missing values in both the variables.

During pre-processing 99 duplicates of the news articles were deleted. No instances of articles with conflicting category labels were found. The cleaned data set had 2,126 unique articles.

In the cleaned data set the number of articles by category was Business (503), Entertainment (369), Politics (403), Sport (504) and Tech (347). Each news article had an average of around 390 words. The minimum and maximum lengths of articles were 90 and 4,492 respectively.

The cleaned data was split into training and testing sets using 80:20 ratio. The split resulted in 1,700 articles for training and 426 articles for testing.

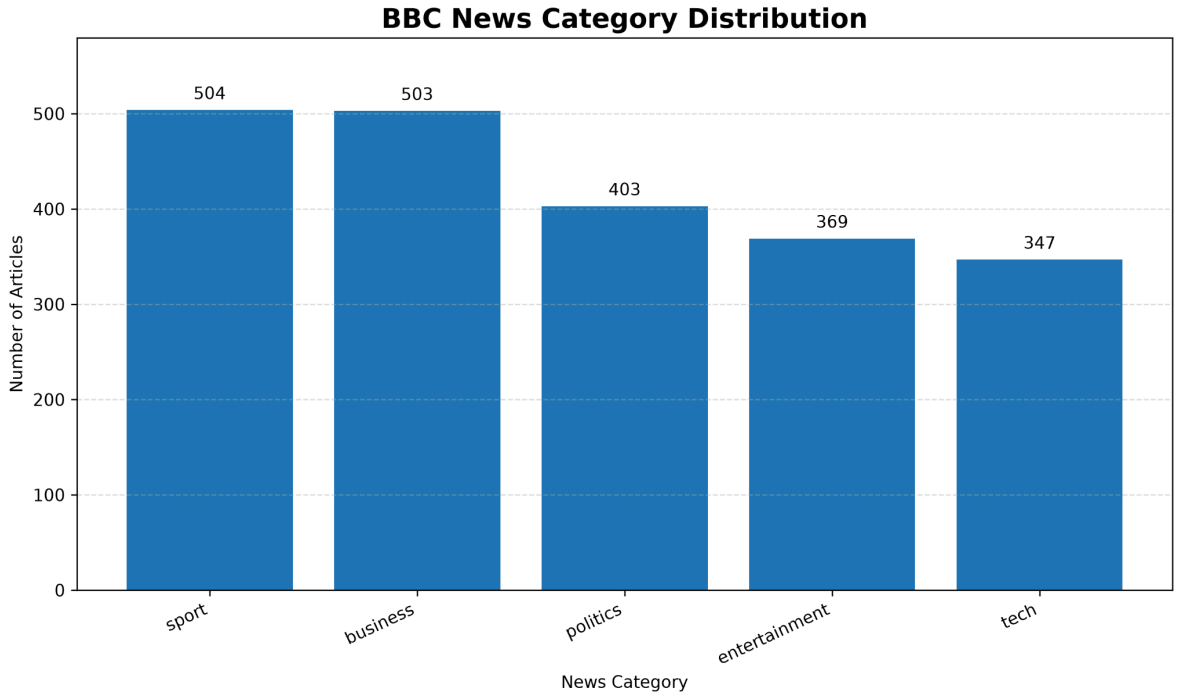
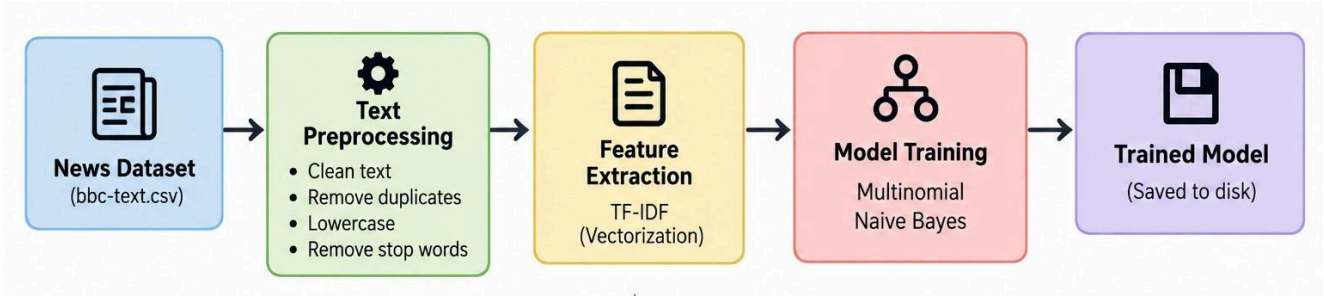


Fig. BBC News Category Distribution

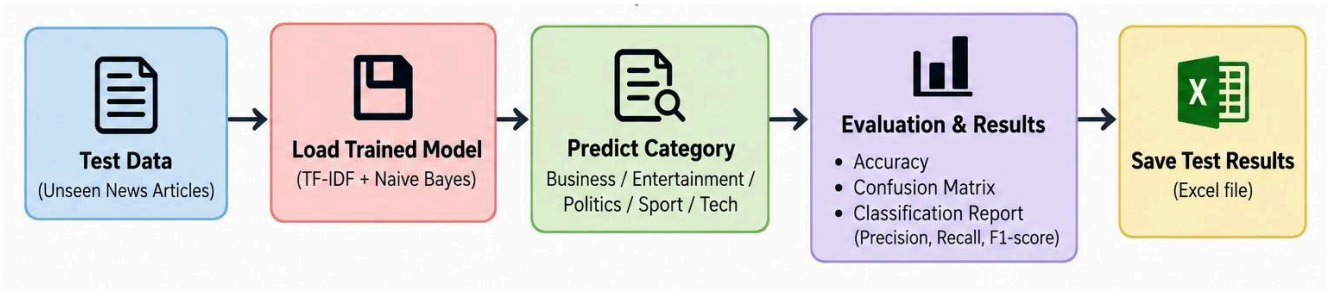
9. NLP Methodology / System Architecture

Draw or insert the system architecture / workflow diagram below.

Training Phase:



Testing Phase:



Workflow steps:

1. Load the BBC News dataset from the CSV file.
2. Check for the presence of the needed columns and drop missing records.
3. Drop duplicates and records that have contradicting labels, if any.
4. Split the processed data into a train and test set with 80% to 20% stratified split.
5. Extract the TF-IDF features for the text.
6. Train the Multinomial Naive Bayes classifier on the training dataset.
7. Save the whole training pipeline and test data separately.
8. Load the saved model, do predictions for the testing articles, evaluate the model performance, create visualizations, and save the test results into Excel.

10. Text Preprocessing

Preprocessing Step	Method / Tool Used	Observation
Data cleaning	Pandas; remove missing, empty, duplicate, and conflicting records	No missing or conflicting-label records; 99 duplicate texts removed
Tokenization	TfidfVectorizer default word tokenization	Article text converted into word-based features
Normalization	Lowercasing and whitespace trimming	Text is processed consistently
Stop-word handling	stop_words='english'	Common English stop words are excluded

Stemming / Lemmatization	Not applied	Original word forms are retained
Other	TF-IDF with unigrams and bigrams; maximum 20,000 features	Important single words and two-word combinations are represented

11. Feature Representation / NLP Model

Selected representation/model: TF-IDF (Term Frequency-Inverse Document Frequency) for text representation and Multinomial Naive Bayes for classification.

Reason for selection:

TF-IDF represents the text data in numerical form which is then usable by the machine learning classifier. It puts more emphasis on terms that are useful in differentiating the documents. The Multinomial Naive Bayes algorithm is an appropriate choice for classification problems where text features are involved. It can find patterns corresponding to different classes of news.

Important parameters / configuration:

Lowercase mapping, stop word removal for English language, maximum number of features = 20,000, and n-gram range = (1,2) have been used in TF-IDF. Sublinear Term frequency scaling has been turned on. Multinomial Naive Bayes model uses alpha = 0.5. Split ratio for training and testing data = 80:20 with random state = 42.

12. Tools, Libraries and Environment

Sr. No.	Tool / Library	Version	Purpose
1	Python	3.13.5	Programming and execution
2	Pandas	2.2.3	Loading, cleaning, and managing dataset records
3	Scikit-learn	1.8.0	TF-IDF, train-test split, Naive Bayes, and evaluation
4	NumPy	2.3.5	Numerical calculations and array handling
5	Joblib	1.5.3	Saving and loading the trained model
6	Matplotlib	3.10.8	Creating confusion matrix and

			classification report plots
7	OpenPyXL	3.1.5	Writing and formatting Excel output
8	Pathlib	Python standard library	File paths and directory management

13. Implementation / Source Code

The implementation is provided separately in two Python scripts: train_model.py for preprocessing, model training, and saving; and test_model.py for model testing, evaluation, visualization, and result export.

14. Experimental Setup

Parameter	Value / Configuration	Remarks
Train/Test split	80:20	1,700 training records and 426 testing records
Model	TF-IDF + Multinomial Naive Bayes	Five-category classification
Hyperparameters	alpha=0.5; max_features=20000; ngram_range=(1,2)	TF-IDF uses English stop-word removal and sublinear term-frequency scaling
Batch size	Not applicable	Traditional machine learning model; no neural-network batches
Number of epochs / iterations	One model-fitting operation	The model is trained using fit()
Random seed	42	Used for a reproducible data split
Hardware	CPU-based execution; no GPU required	Record your actual computer specifications if required by your faculty
Other	Stratified train-test splitting	Preserves category proportions as closely as possible

15. Evaluation Metrics

Metric	Value	Interpretation

Accuracy	96.95%	413 of 426 articles were classified correctly

Precision	97.50% macro average	Average precision across the five categories
Recall	96.46% macro average	Average recall across the five categories
F1-score	96.88% macro average	Harmonic mean of macro precision and macro recall
Other	96.97% weighted F1-score	F1-score weighted by the number of test articles in each category

16. Results and Output

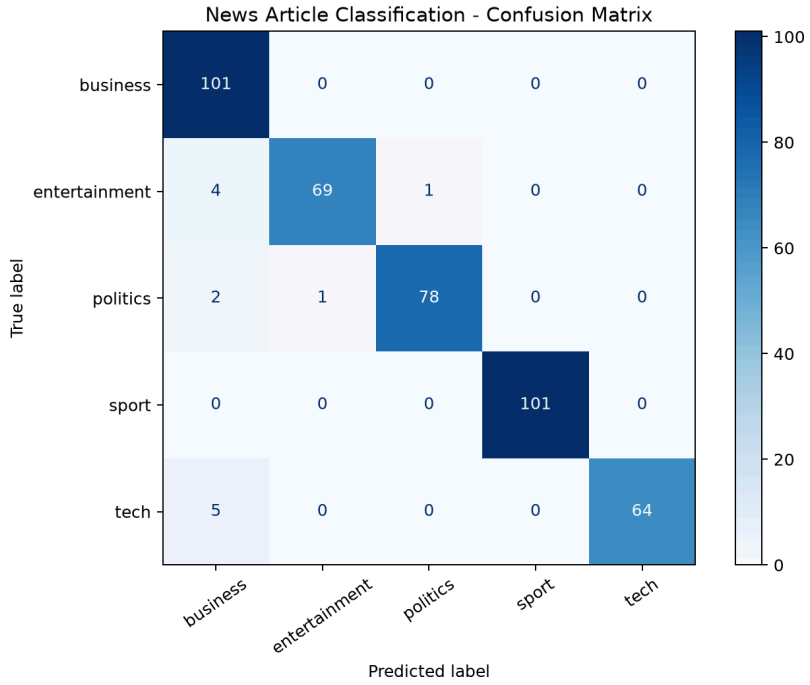
The accuracy of the news classification model was tested on 426 test articles that were not used during training. The model has made correct predictions for 413 articles and incorrect ones for 13. The accuracy is 96.95%, macro precision – 97.50%, macro recall – 96.46%, and macro F1 score – 96.88%.

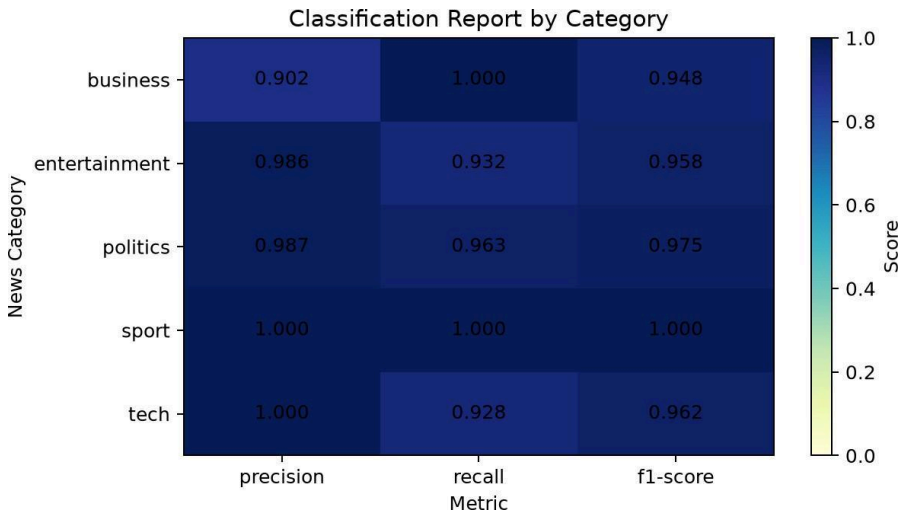
Confusion matrix represents the number of true positive and false positive predictions for each class. Classification report represents the precision, recall, F1 score, and support for each class. Testing results are stored in the Excel workbook as well.

```
=== Classification Report ===
               precision    recall  f1-score   support

     business       0.95      0.97      0.96       100
entertainment       1.00      0.93      0.96        73
     politics       0.95      0.99      0.97        79
        sport       1.00      0.99      0.99        99
         tech       0.96      0.97      0.96        68

     accuracy                           0.97       419
    macro avg       0.97      0.97      0.97       419
 weighted avg       0.97      0.97      0.97       419
```





17. Result Analysis

The accuracy of classification reached 96.95%, where 413 articles were classified accurately out of the total 426 articles. The Macro Precision and Macro Recall values were calculated as 97.50% and 96.46% respectively.

The precision, recall and F1-Score values for Sport category reached 1.00 in this experiment. The recall value for Business category reached 1.00, however, its precision was only 0.9018 because of misclassifications of some articles from other categories as Business. Some Entertainment and Tech articles were misclassified too.

This demonstrates that TF-IDF and Multinomial Naive Bayes algorithm combination performed very well with this dataset. However, the results are valid for this dataset only and cannot be generalized for future use.

18. Error Analysis

Identify representative incorrect predictions or system failures and explain possible causes.

Input / Example	Expected	Predicted	Reason / Analysis
“Argonaut founder rebuilds empire”	Tech	Business	The article contains terms related to cash, losses, and a company, which may resemble business news.
“Anti-spam laws bite spammer hard”	Tech	Business	References to lawsuits, bankruptcy protection, and trading may overlap with business-related vocabulary.
“UK world's biggest music buyer”	Entertainment	Business	Terms about sales, the music industry, and global markets may lead to confusion with business news.
“UK's National Gallery in the pink”	Entertainment	Business	The article discusses visitor numbers and financial-style figures, which may resemble business reporting.
“Games maker fights for survival”	Tech	Business	Discussion of a game company, cash problems, and staff losses may cause overlap between technology and business vocabulary.

19. Advantages and Limitations

Advantages:

1. The system automatically classifies news articles into five predefined categories.
2. TF-IDF converts text into numerical features suitable for machine learning.
3. Multinomial Naive Bayes is straightforward to train and use for text classification.
4. The trained model and evaluation results can be saved and

reused. Limitations:

1. The model can confuse articles that use similar words across different categories.

2. TF-IDF does not fully understand the meaning or context of sentences.
3. The model is trained on a particular dataset and may not perform equally well on newer news articles.
4. The system classifies topics but does not determine whether a news article is true or false.

20. Ethical / Practical Considerations

News articles contained in the dataset have been taken from a source that retains copyright over the information published. The dataset must be used for research and non-commercial purposes only.

It is possible that the classifier will make mistakes, which means that the results of its work cannot be considered to be entirely accurate. This can happen because different news categories can employ similar terminology.

The dataset includes articles from a certain source and time period; therefore, the model does not capture all news sources, topics, and ways of writing. The system is able to determine the category an article belongs to but not whether this article is true or not.

The system must be tested before being used in practice in order to check for accuracy of its work.

21. Future Scope

Some improvements that can be made to the system include increasing diversity in the dataset by using more recent news articles from various sources. Other methods of pre-processing and settings for TF-IDF can be experimented with to see if it would improve the classification results.

Machine Learning algorithms other than the ones already used, like Logistic Regression and Support Vector Machines, can also be tried out to see how their classification performance compares against Multinomial Naive Bayes. Cross-validation can be employed to test performance on other datasets.

A GUI-based application can be implemented where the user can input the article and receive the result of classification.

22. Conclusion

News Article Classification using Natural Language Processing and Machine Learning was performed in this project. Cleaning of BBC News dataset and transforming it to numerical format by using the TF-IDF method were carried out. Then a Multinomial Naive Bayes classifier was trained on the processed data.

The classifier obtained accuracy score of 96.95% for 426 test news articles and macro precision score of 97.50%, macro recall score of 96.46% and macro F1-score of 96.88%.

The project is an example of using text representation techniques and machine learning algorithms for automatic classification of news articles. Although the classifier was working effectively on a selected test set, there are cases when it makes wrong classifications and needs additional tests.

23. Viva Questions

1. What is the NLP problem addressed by your case study?

Answer: The project classifies news articles into Business, Entertainment, Politics, Sport, and Tech categories using NLP and machine learning.

2. Why is NLP suitable for this application?

Answer: News articles contain text that can be processed to identify useful word patterns and predict their topics automatically.

3. What dataset did you use and why?

Answer: I used the BBC Full Text and Category dataset because it contains labelled news articles suitable for supervised text classification.

4. What preprocessing steps did you apply?

Answer: I removed missing or empty records, checked for conflicting labels, removed duplicate articles, converted text to lowercase, and used English stop-word removal during TF-IDF vectorization.

5. Why did you select the particular model/algorithm?

Answer: TF-IDF represents important words numerically, and Multinomial Naive Bayes is a straightforward machine learning classifier suitable for text classification.

6. What evaluation metrics did you use and why?

Answer: I used accuracy, precision, recall, and F1-score to measure overall correctness and classification performance across the five categories. I also used a confusion matrix to inspect prediction errors.

7. What are the major sources of error in your system?

Answer: Articles from different categories may contain similar words. For example, technology news about a company's financial problems may be mistaken for business news.

8. What are the limitations of your implementation?

Answer: The model depends on the training dataset, may confuse categories with similar vocabulary, and does not fully understand sentence meaning or verify whether an article is factually correct.

9. How can the system be improved?

Answer: It can be improved by adding newer and more diverse articles, tuning TF-IDF parameters, comparing other classifiers, and developing a user interface.

10. What ethical or practical issues should be considered?

Answer: The dataset's copyright and usage terms must be respected. Predictions may be incorrect, so the model should be tested carefully and its output should not be treated as guaranteed to be correct.

24. References

1. Greene, D. and Cunningham, P. (2006). Practical Solutions to the Problem of Diagonal Dominance in Kernel Document Clustering. Proceedings of the 23rd International Conference on Machine Learning (ICML). https://derekgreene.com/papers/greene06icml.pdf

2. Greene, Derek. BBC Datasets — News Article Datasets for Machine Learning Research. http://mlg.ucd.ie/datasets/bbc.html

3. Yufeng. BBC Text Categorization — BBC Full Text and Category Dataset. Kaggle. https://www.kaggle.com/code/yufengdev/bbc-text-categorization/data

4. Scikit-learn Developers. TfidfVectorizer Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

5. Scikit-learn Developers. MultinomialNB Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html

6. Scikit-learn Developers. Model Evaluation: Classification Report and Confusion Matrix. https://scikit-learn.org/stable/modules/model_evaluation.html

25. Faculty Evaluation

Evaluation Parameter	Marks	Remarks
Problem definition and motivation	--/10	
Dataset and preprocessing	--/20	
NLP methodology / model	--/20	
Implementation correctness	--/20	
Evaluation and analysis	--/10	
Documentation / presentation	--/10	
Viva / demonstration	--/10	
Total	--/100	

Student Signature: _____

Faculty Signature: _____

Date: _____

Source Code

Code:

> train_model.py

```
from pathlib import Path
import json

import joblib
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB


# PATH CONFIGURATION

BASE_DIR = Path(_file_).resolve().parent

DATASET_PATH = BASE_DIR / "dataset" / "bbc-text.csv"

ARTIFACT_DIR = BASE_DIR / "artifacts"
VISUALIZATION_DIR = ARTIFACT_DIR / "visualization"

MODEL_PATH = ARTIFACT_DIR / "news_classifier.joblib"
TEST_DATA_PATH = ARTIFACT_DIR / "test_data.csv"
METADATA_PATH = ARTIFACT_DIR / "training_metadata.json"

TEST_SIZE = 0.20
RANDOM_STATE = 42


# DATASET LOADING AND CLEANING

def load_and_preprocess_dataset(path: Path) -> tuple[pd.DataFrame, dict]:
    if not path.exists():
        raise FileNotFoundError(
            f"Dataset not found: {path}\n"
            "Place bbc-text.csv inside the dataset folder."
        )
```

```
df = pd.read_csv(path)

original_rows = len(df)

# Standardize column names.
df.columns = [str(column).strip().lower() for column in df.columns]

required_columns = {"text", "category"}
missing_columns = required_columns - set(df.columns)

if
   missing_columns
   : raise
   ValueError(
      f"Dataset is missing required columns: {sorted(missing_columns)}. "
      f"Found columns: {list(df.columns)}"
   )

df = df[["text", "category"]].copy()

# Remove missing and empty values.
df = df.dropna(subset=["text", "category"])

df["text"] = df["text"].astype(str).str.strip()
df["category"] = (
   df["category"].astype(str).str.strip().str.lower()
)

df = df[
   (df["text"] != "") &
   (df["category"] != "")
]

rows_after_missing_cleanup = len(df)

# Remove articles whose identical text has conflicting labels.
label_counts_per_text = df.groupby("text")["category"].nunique()

ambiguous_texts = label_counts_per_text[
   label_counts_per_text > 1
].index

ambiguous_rows_removed = int(
```

```
df["text"].isin(ambiguous_texts).sum()
```

```
)

if len(ambiguous_texts) > 0:
    df = df[~df["text"].isin(ambiguous_texts)]

# Remove duplicate articles.
rows_before_deduplication = len(df)

df = df.drop_duplicates(
    subset=["text"],
    keep="first"
).reset_index(drop=True)

duplicate_rows_removed = (
    rows_before_deduplication - len(df)
)

# Validate the dataset for stratified splitting.
if df["category"].nunique() < 2:
    raise ValueError(
        "At least two categories are required to train a classifier."
    )

if df["category"].value_counts().min() < 2:
    raise ValueError(
        "Every category must have at least two rows "
        "for a stratified split."
    )

cleaning_stats = {
    "original_rows": int(original_rows),
    "rows_after_missing_or_empty_cleanup": int(
        rows_after_missing_cleanup
    ),
    "ambiguous_text_rows_removed":
        int( ambiguous_rows_removed
    ),
    "duplicate_text_rows_removed": int(
        duplicate_rows_removed
    ),
    "cleaned_rows": int(len(df)),
}
```

```
    return df, cleaning_stats


# ------------------------------
# VISUALIZATION FUNCTIONS
# ------------------------------

def save_cleaning_summary(
    cleaning_stats: dict,
    output_dir: Path
) -> None:

    labels = [
        "Original Rows",
        "After Missing /\nEmpty Cleanup",
        "Ambiguous Rows\nRemoved",
        "Duplicate Rows\nRemoved",
        "Cleaned Rows",
    ]

    values = [
        cleaning_stats["original_rows"],
        cleaning_stats["rows_after_missing_or_empty_cleanup"],
        cleaning_stats["ambiguous_text_rows_removed"],
        cleaning_stats["duplicate_text_rows_removed"],
        cleaning_stats["cleaned_rows"],
    ]
    fig, ax = plt.subplots(figsize=(12,

    6)) bars = ax.bar(labels, values)

    ax.set_title(

        "BBC News Dataset Cleaning Summary",
        fontsize=16,
        fontweight="bold"
    )
    ax.set_ylabel("Number of Rows")
    ax.set_xlabel("Cleaning Stage")
    ax.grid(axis="y", linestyle="--", alpha=0.4)

    ax.bar_label(bars, fmt="%d", padding=4)
```

```
    ax.set_ylim(0, max(values, default=1) * 1.15)

    plt.tight_layout()

    output_path = output_dir / "cleaning_summary.png"
    fig.savefig(output_path, dpi=300, bbox_inches="tight")
    plt.close(fig)

    print(f"Saved cleaning summary chart: {output_path}")


def
    save_category_distribution(
    df: pd.DataFrame,
    output_dir: Path
) -> None:

    category_counts = df["category"].value_counts()

    fig, ax = plt.subplots(figsize=(10, 6))

    bars = ax.bar(
        category_counts.index,
        category_counts.values
    )

    ax.set_title(
        "BBC News Category Distribution",
        fontsize=16,
        fontweight="bold"
    )
    ax.set_xlabel("News Category")
    ax.set_ylabel("Number of Articles")
    ax.grid(axis="y", linestyle="--", alpha=0.4)

    ax.bar_label(bars, fmt="%d", padding=4)

    ax.set_ylim(0, category_counts.max() * 1.15)
    plt.xticks(rotation=25, ha="right")

    plt.tight_layout()

    output_path = output_dir / "category_distribution.png"
    fig.savefig(output_path, dpi=300, bbox_inches="tight")
```

```
    plt.close(fig)

    print(f"Saved category distribution chart: {output_path}")


def save_category_pie_chart(
    df: pd.DataFrame,
    output_dir: Path
) -> None:

    category_counts = df["category"].value_counts()

    fig, ax = plt.subplots(figsize=(9, 7))

    ax.pie(
        category_counts.values,
        labels=category_counts.index,
        autopct="%1.1f%%",
        startangle=90,
        pctdistance=0.75
    )

    ax.set_title(
        "BBC News Category Proportions",
        fontsize=16,
        fontweight="bold"
    )

    plt.tight_layout()

    output_path = output_dir / "category_distribution_pie.png"
    fig.savefig(output_path, dpi=300, bbox_inches="tight")
    plt.close(fig)

    print(f"Saved category pie chart: {output_path}")


def
    save_dataset_overview(
    cleaning_stats: dict,
    df: pd.DataFrame,
    output_dir: Path
) -> None:
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Cleaning
summary.
cleaning_labels = [
    "Original",
    "After Missing /\nEmpty
    Cleanup",
    "Ambiguous\nRemoved",
    "Duplicates\nRemoved",
    "Cleaned",
]

cleaning_values = [
    cleaning_stats["original_rows"],
    cleaning_stats["rows_after_missing_or_empty_cleanup"],
    cleaning_stats["ambiguous_text_rows_removed"],
    cleaning_stats["duplicate_text_rows_removed"],
    cleaning_stats["cleaned_rows"],
]

cleaning_bars = axes[0].bar(
    cleaning_labels,
    cleaning_values
)

axes[0].set_title("Dataset Cleaning")
axes[0].set_ylabel("Number of Rows")
axes[0].grid(axis="y", linestyle="--", alpha=0.4)
axes[0].bar_label(cleaning_bars, fmt="%d", padding=3)
axes[0].tick_params(axis="x", labelrotation=20)

# Category distribution.
category_counts = df["category"].value_counts()

category_bars = axes[1].bar(
    category_counts.index,
    category_counts.values
)

axes[1].set_title("Category Distribution")
axes[1].set_xlabel("Category")
axes[1].set_ylabel("Number of Articles")
axes[1].grid(axis="y", linestyle="--", alpha=0.4)
```

```
axes[1].bar_label(category_bars, fmt="%d", padding=3)
```

```
    axes[1].tick_params(axis="x", labelrotation=25)

    fig.suptitle(
        "BBC News Dataset Overview",
        fontsize=17,
        fontweight="bold"
    )

    plt.tight_layout()

    output_path = output_dir / "dataset_overview.png"
    fig.savefig(output_path, dpi=300, bbox_inches="tight")
    plt.close(fig)

    print(f"Saved dataset overview dashboard: {output_path}")


def generate_visualizations(
    cleaning_stats: dict,
    df: pd.DataFrame
) -> None:
    VISUALIZATION_DIR.mkdir(parents=True, exist_ok=True)

    save_cleaning_summary(

        cleaning_stats,
        VISUALIZATION_DIR
    )

    save_category_distribution(
        df,
        VISUALIZATION_DIR
    )

    save_category_pie_chart(
        df,
        VISUALIZATION_DIR
    )

    save_dataset_overview(
        cleaning_stats,
        df,
        VISUALIZATION_DIR
```

```
    )


# ------------------------------
# MODEL TRAINING
# ------------------------------

def main() -> None:
    ARTIFACT_DIR.mkdir(parents=True, exist_ok=True)
    VISUALIZATION_DIR.mkdir(parents=True, exist_ok=True)

    # Load and clean the dataset.
    df, cleaning_stats = load_and_preprocess_dataset(
        DATASET_PATH
    )

    X = df["text"]
    y = df["category"]

    # Split data while preserving category proportions.
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=TEST_SIZE
        ,
        random_state=RANDOM_STATE,
        stratify=y,
    )

    # TF-IDF and Naive Bayes pipeline.
    model = Pipeline(
        steps=[
            (
                "tfidf",
                TfidfVectorizer(
                    lowercase=True,
                    stop_words="english",
                    max_features=20000,
                    ngram_range=(1, 2),
                    sublinear_tf=True,
                ),
            ),
            (
                "classifier",
```

```
            MultinomialNB(alpha=0.5),
        ),
    ]
)

print("\nTraining the TF-IDF + Multinomial Naive Bayes model...")

model.fit(X_train, y_train)

# Save the trained pipeline.
joblib.dump(model, MODEL_PATH)

# Save the held-out test dataset.
test_df = pd.DataFrame({
    "text": X_test.to_numpy(),
    "category": y_test.to_numpy(),
})

test_df.to_csv(
    TEST_DATA_PATH,
    index=False,
    encoding="utf-8"
)

# Generate visualizations.
generate_visualizations(
    cleaning_stats,
    df
)

# Save training metadata.
metadata = {
    "dataset_file": DATASET_PATH.name,
    "model_file": MODEL_PATH.name,
    "test_data_file": TEST_DATA_PATH.name,
    "algorithm": "TF-IDF + Multinomial Naive Bayes",
    "tfidf_parameters": {
        "lowercase": True,
        "stop_words": "english",
        "max_features": 20000,
        "ngram_range": [1, 2],
        "sublinear_tf": True,
    },
```

```
    "naive_bayes_alpha": 0.5,
    "test_size": TEST_SIZE,
    "random_state": RANDOM_STATE,
    "training_rows": int(len(X_train)),
    "testing_rows": int(len(X_test)),
    "categories": sorted(y.unique().tolist()),
    "cleaning_stats": cleaning_stats,
    "category_distribution_after_cleaning": {
        str(category):
        int(count) for category,
        count in
        df["category"].value_counts().items()
    },
    "visualization_directory": str(
        VISUALIZATION_DIR.relative_to(BASE_DIR)
    ),
}

METADATA_PATH.write_text(
    json.dumps(metadata, indent=2),
    encoding="utf-8"
)

# Print cleaning summary.
print("\nDataset Cleaning Summary:")

for key, value in cleaning_stats.items():
    formatted_key = key.replace("_", " ").title()
    print(f" {formatted_key}: {value}")

# Print category counts.
print("\nCategory Counts After Cleaning:")
print(df["category"].value_counts().to_string())

# Print split sizes.
print(f"\nTraining Rows: {len(X_train)}")
print(f"Testing Rows: {len(X_test)}")

# Print saved artifact paths.
print(f"\nSaved Trained Model: {MODEL_PATH}")
print(f"Saved Held-Out Test Data: {TEST_DATA_PATH}")
print(f"Saved Training Metadata: {METADATA_PATH}")
print(f"Saved Visualizations: {VISUALIZATION_DIR}")
```

```
    print("\nGenerated Visualization Files:")
    for image_path in sorted(VISUALIZATION_DIR.glob("*.png")):
        print(f" {image_path.name}")


if __name__ == "__main__":
    main()
```

> test_model.py

```
from pathlib import Path

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    ConfusionMatrixDisplay,
    precision_score,
    recall_score,
    f1_score,
)

BASE_DIR = Path(__file__).resolve().parent
ARTIFACT_DIR = BASE_DIR / "artifacts"
MODEL_PATH = ARTIFACT_DIR / "news_classifier.joblib"
TEST_DATA_PATH = ARTIFACT_DIR / "test_data.csv"
RESULTS_DIR = BASE_DIR / "results"
EXCEL_PATH = RESULTS_DIR / "testing_results.xlsx"
CONFUSION_MATRIX_PATH = RESULTS_DIR / "confusion_matrix.png"
CLASSIFICATION_REPORT_PATH = RESULTS_DIR / "classification_report.png"


def style_excel_file(path: Path) -> None:
    from openpyxl import load_workbook
    from openpyxl.styles import Alignment, Font, PatternFill
    from openpyxl.utils import get_column_letter

    workbook = load_workbook(path)
    header_fill = PatternFill("solid", fgColor="1F4E78")
```

```
    header_font = Font(bold=True, color="FFFFFF")

    for worksheet in workbook.worksheets:
        worksheet.freeze_panes = "A2"
        worksheet.auto_filter.ref = worksheet.dimensions

        for cell in worksheet[1]:
            cell.fill = header_fill
            cell.font = header_font
            cell.alignment = Alignment(vertical="top", wrap_text=True)

        for row in worksheet.iter_rows(min_row=2):
            for cell in row:
                cell.alignment = Alignment(vertical="top", wrap_text=True)

        # Worksheet-specific widths and row formatting.
        if worksheet.title == "Test Results":
            widths = {"A": 90, "B": 22, "C": 22, "D": 18}
            for row_index in range(2, worksheet.max_row + 1):
                worksheet.row_dimensions[row_index].height = 48
        elif worksheet.title == "Metrics":
            widths = {"A": 30, "B": 20}
        else:
            widths = {"A": 24, "B": 16, "C": 16, "D": 16, "E": 16}

        for column_letter, width in widths.items():
            worksheet.column_dimensions[column_letter].width = width

        worksheet.sheet_view.showGridLines =

    False workbook.save(path)


def save_excel_results(
    results_df: pd.DataFrame,
    metrics: dict,
    report_dict: dict,
    path: Path,
) -> None:
    report_without_accuracy = {
        key: value for key, value in report_dict.items() if key != "accuracy"
    }
    report_df = pd.DataFrame(report_without_accuracy).T
```

```
    report_df.index.name = "Category / Average"
    report_df = report_df.reset_index()

    metrics_df = pd.DataFrame(
        [{"Metric": metric_name, "Value": metric_value}
         for metric_name, metric_value in metrics.items()]
    )

    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        # This sheet has the exact four columns requested by the assignment.
        results_df.to_excel(writer, sheet_name="Test Results", index=False)
        metrics_df.to_excel(writer, sheet_name="Metrics", index=False)
        report_df.to_excel(writer, sheet_name="Classification Report", index=False)

    style_excel_file(path)


def save_classification_report_plot(report_dict: dict, path: Path) -> None:
    class_rows = {
        label: values
        for label, values in report_dict.items()
        if isinstance(values, dict) and label not in {"macro avg", "weighted avg"}
    }
    metrics_names = ["precision", "recall", "f1-score"]
    labels = list(class_rows.keys())
    matrix = np.array(
        [[class_rows[label][metric] for metric in metrics_names] for label in labels],
        dtype=float,
    )

    fig, ax = plt.subplots(figsize=(7, max(4, len(labels) * 0.8)))
    image = ax.imshow(matrix, cmap="YlGnBu", vmin=0, vmax=1, aspect="auto")
    ax.set_xticks(range(len(metrics_names)), metrics_names)
    ax.set_yticks(range(len(labels)), labels)
    ax.set_title("Classification Report by Category")
    ax.set_xlabel("Metric")
    ax.set_ylabel("News Category")

    for row_index in range(matrix.shape[0]):
        for col_index in range(matrix.shape[1]):
            ax.text(
                col_index,
                row_index,
```

```
            f"{matrix[row_index, col_index]:.3f}",
            ha="center",
            va="center",
            color="black",
        )

    fig.colorbar(image, ax=ax, label="Score")
    fig.tight_layout()
    fig.savefig(path, dpi=180, bbox_inches="tight")
    return fig


def main() -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)

    if not MODEL_PATH.exists() or not TEST_DATA_PATH.exists():
        raise FileNotFoundError(
            "Required model/test data is missing. Run train_model.py first."
        )

    model = joblib.load(MODEL_PATH)
    test_df =
    pd.read_csv(TEST_DATA_PATH)
    required_columns = {"text", "category"}
    if not required_columns.issubset(test_df.columns):
        raise ValueError(f"Test file must contain columns: {sorted(required_columns)}")
    if test_df.empty:
        raise ValueError("The test dataset is empty.")

    X_test = test_df["text"].fillna("").astype(str)
    y_true = test_df["category"].astype(str)
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_true, y_pred)
    macro_precision = precision_score(y_true, y_pred, average="macro", zero_division=0)
    macro_recall = recall_score(y_true, y_pred, average="macro", zero_division=0)
    macro_f1 = f1_score(y_true, y_pred, average="macro", zero_division=0)
    weighted_precision = precision_score(y_true, y_pred, average="weighted", zero_division=0)
    weighted_recall = recall_score(y_true, y_pred, average="weighted", zero_division=0)
    weighted_f1 = f1_score(y_true, y_pred, average="weighted", zero_division=0)

    # Build the Excel prediction sheet with the exact requested column names/order.
    results_df = pd.DataFrame(
        {
```

```
        "Text": X_test,
        "Actual Category": y_true,
        "Predicted Category": y_pred,
        "Result (Pass/Fail)": np.where(y_true.to_numpy() == y_pred, "Pass", "Fail"),
    }
)

labels = sorted(set(y_true.tolist()) | set(y_pred.tolist()))
report_text = classification_report(
    y_true,
    y_pred,
    labels=labels,
    zero_division=0,
    digits=4,
)
report_dict = classification_report(
    y_true,
    y_pred,
    labels=labels,
    zero_division=0,
    output_dict=True,
)

metrics = {
    "Accuracy": round(float(accuracy), 6),
    "Macro Precision": round(float(macro_precision), 6),
    "Macro Recall": round(float(macro_recall), 6),
    "Macro F1-score": round(float(macro_f1), 6),
    "Weighted Precision": round(float(weighted_precision), 6),
    "Weighted Recall": round(float(weighted_recall), 6),
    "Weighted F1-score": round(float(weighted_f1), 6),
    "Total Test Samples": int(len(y_true)),
    "Correct Predictions": int((y_true.to_numpy() == y_pred).sum()),
    "Incorrect Predictions": int((y_true.to_numpy() != y_pred).sum()),
}

save_excel_results(results_df, metrics, report_dict, EXCEL_PATH)

# Save a color-coded confusion matrix image and show it on screen.
fig, ax = plt.subplots(figsize=(8, 6))
ConfusionMatrixDisplay.from_predictions(
    y_true,
    y_pred,
```

```
        labels=labels,
        display_labels=labels
        , cmap="Blues",
        xticks_rotation=35,
        values_format="d",
        ax=ax,
        colorbar=True,
    )
    ax.set_title("News Article Classification - Confusion Matrix")
    fig.tight_layout()
    fig.savefig(CONFUSION_MATRIX_PATH, dpi=180, bbox_inches="tight")
    plt.show()
    plt.close(fig)

    # Save and display a second heatmap for per-category precision, recall and F1.
    report_fig = save_classification_report_plot(
        report_dict, CLASSIFICATION_REPORT_PATH
    )
    plt.show()
    plt.close(report_fig)

    print("\n========== EVALUATION RESULTS ==========")
    print(f"Accuracy:          {accuracy:.4f} ({accuracy * 100:.2f}%)")
    print(f"Macro Precision: {macro_precision:.4f}")
    print(f"Macro Recall:      {macro_recall:.4f}")
    print(f"Macro F1-score:    {macro_f1:.4f}")
    print(f"Weighted Precision:{weighted_precision:.4f}")
    print(f"Weighted Recall: {weighted_recall:.4f}")
    print(f"Weighted F1-score: {weighted_f1:.4f}")
    print(f"Correct predictions: {metrics['Correct Predictions']}")
    print(f"Incorrect predictions: {metrics['Incorrect Predictions']}")

    print("\n========== CLASSIFICATION REPORT ==========")
    print(report_text)

    print(f"Excel testing results: {EXCEL_PATH}")
    print(f"Confusion matrix image: {CONFUSION_MATRIX_PATH}")
    print(f"Classification report heatmap: {CLASSIFICATION_REPORT_PATH}")
    print("Excel workbook contains: Test Results, Metrics, and Classification Report sheets.")


if __name__ == "__main__":
    main()
```

Execution:

```
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 6$ python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
Requirement already satisfied: pip in ./.venv/lib/python3.13/site-packages (25.2)
Collecting pip
  Downloading pip-26.2.2-py3-none-any.whl.metadata (4.6 kB)
Downloading pip-26.2.2-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 4.4 MB/s  0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.2
    Uninstalling pip-25.2:
      Successfully uninstalled pip-25.2
Successfully installed pip-26.2.2
Collecting pandas>=2.0.0 (from -r requirements.txt (line 1))
  Downloading pandas-3.1.0-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (18 kB)
Collecting numpy>=1.24.0 (from -r requirements.txt (line 2))
  Downloading numpy-2.5.4-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting scikit-learn>=1.3.0 (from -r requirements.txt (line 3))
  Downloading scikit_learn-1.9.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.3 kB)
Collecting matplotlib>=3.7.0 (from -r requirements.txt (line 4))
  Downloading matplotlib-3.11.2-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (80 kB)
Collecting seaborn>=0.12.0 (from -r requirements.txt (line 5))
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Collecting joblib>=1.3.0 (from -r requirements.txt (line 6))
  Downloading joblib-1.6.0-py3-none-any.whl.metadata (6.5 kB)
Collecting openpyxl>=3.1.0 (from -r requirements.txt (line 7))
  Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
Collecting python-dateutil>=2.9.0 (from pandas>=2.0.0->-r requirements.txt (line 1))
  Downloading python_dateutil-3.0.0-py2.py3-none-any.whl.metadata (8.2 kB)
Collecting scipy>=1.10.0 (from scikit-learn>=1.3.0->-r requirements.txt (line 3))
  Downloading scipy-1.18.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (62 kB)
Collecting narwhals>=2.0.1 (from scikit-learn>=1.3.0->-r requirements.txt (line 3))
  Downloading narwhals-2.27.1-py3-none-any.whl.metadata (15 kB)
Collecting threadpoolctl>=3.5.0 (from scikit-learn>=1.3.0->-r requirements.txt (line 3))
  Downloading threadpoolctl-3.7.0-py3-none-any.whl.metadata (24 kB)
Collecting contourpy>=1.0.1 (from matplotlib>=3.7.0->-r requirements.txt (line 4))
  Downloading contourpy-1.4.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (3.8 kB)
Collecting cycler>=0.10 (from matplotlib>=3.7.0->-r requirements.txt (line 4))
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.28.2 (from matplotlib>=3.7.0->-r requirements.txt (line 4))
  Downloading fonttools-4.67.0-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (137 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib>=3.7.0->-r requirements.txt (line 4))
  Downloading kiwisolver-1.5.1-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (5.2 kB)
Collecting packaging>=20.0 (from matplotlib>=3.7.0->-r requirements.txt (line 4))
  Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pillow>=9 (from matplotlib>=3.7.0->-r requirements.txt (line 4))
```

```
Collecting kiwisolver>=1.3.1 (from matplotlib>=3.7.0->-r requirements.txt (line 4))
  Downloading kiwisolver-1.5.1-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (5.2 kB)
Collecting packaging>=20.0 (from matplotlib>=3.7.0->-r requirements.txt (line 4))
  Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pillow>=9 (from matplotlib>=3.7.0->-r requirements.txt (line 4))
  Downloading pillow-13.0.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.1 kB)
Collecting pyparsing>=3 (from matplotlib>=3.7.0->-r requirements.txt (line 4))
  Downloading pyparsing-3.3.3-py3-none-any.whl.metadata (5.9 kB)
Collecting cloudpickle>=3.0 (from joblib>=1.3.0->-r requirements.txt (line 6))
  Downloading cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)
Collecting et-xmlfile (from openpyxl>=3.1.0->-r requirements.txt (line 7))
  Downloading et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Collecting six>=1.6 (from python-dateutil>=2.9.0->pandas>=2.0.0->-r requirements.txt (line 1))
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Collecting tzdata (from python-dateutil>=2.9.0->pandas>=2.0.0->-r requirements.txt (line 1))
  Downloading tzdata-2026.5-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading pandas-3.1.0-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (12.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.0/12.0 MB 3.2 MB/s  0:00:03
Downloading numpy-2.5.4-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.7/16.7 MB 2.6 MB/s  0:00:06
Downloading scikit_learn-1.9.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (9.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.1/9.1 MB 3.2 MB/s  0:00:02
Downloading matplotlib-3.11.2-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (9.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.9/9.9 MB 4.6 MB/s  0:00:02
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
Downloading joblib-1.6.0-py3-none-any.whl (306 kB)
Downloading openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
Downloading contourpy-1.4.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (384 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.67.0-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (5.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.4/5.4 MB 5.5 MB/s  0:00:00
Downloading kiwisolver-1.5.1-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 3.6 MB/s  0:00:00
Downloading narwhals-2.27.1-py3-none-any.whl (483 kB)
Using cached packaging-26.3-py3-none-any.whl (129 kB)
Downloading pillow-13.0.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (7.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.1/7.1 MB 5.1 MB/s  0:00:01
Downloading pyparsing-3.3.3-py3-none-any.whl (126 kB)
Downloading python_dateutil-3.0.0-py2.py3-none-any.whl (94 kB)
Downloading scipy-1.18.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (35.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 35.3/35.3 MB 4.4 MB/s  0:00:07
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading threadpoolctl-3.7.0-py3-none-any.whl (26 kB)
Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Downloading tzdata-2026.5-py2.py3-none-any.whl (347 kB)
Installing collected packages: tzdata, threadpoolctl, six, pyparsing, pillow, packaging, numpy, narwhals, kiwisolver, fonttools, et-xmlfile, cycler, cloudpickle, scipy, python-dateutil, openpyxl, joblib, contourp
y, scikit-learn, pandas, matplotlib, seaborn
```

```
Loading and preprocessing dataset...

=== Dataset Cleaning Statistics ===
original_rows: 2225
rows_after_missing_or_empty_cleanup: 2225
ambiguous_text_rows_removed: 0
duplicate_text_rows_removed: 133
cleaned_rows: 2092

=== Cleaned Dataset Category Distribution ===
category
business         500
sport            494
politics         395
entertainment    366
tech             337
Name: count, dtype: int64

Generating visualizations...
Saved cleaning summary chart: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/artifacts/visualization/cleaning_summary.png
Saved category distribution chart: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/artifacts/visualization/category_distribution.png
Saved category pie chart: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/artifacts/visualization/category_distribution_pie.png
Saved dataset overview chart: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/artifacts/visualization/dataset_overview.png

Performing stratified train-test split (80:20, random_state=42)...
Training set size: 1673
Test set size: 419

Building and training pipeline (TF-IDF + MultinomialNB)...
Model training completed.

Saving artifacts...
Saved model: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/artifacts/news_classifier.joblib
Saved test data: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/artifacts/test_data.csv
Saved metadata: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/artifacts/training_metadata.json

Training completed successfully!
(.venv) (base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 6$
```

```
Loading saved model and test data...
Loaded test data with 419 samples

Making predictions...

=== Evaluation Metrics ===
Accuracy: 0.9714 (407/419)
Macro Precision: 0.9717
Macro Recall: 0.9699
Macro F1-Score: 0.9705
Weighted Precision: 0.9720
Weighted Recall: 0.9714
Weighted F1-Score: 0.9714
Correct: 407, Incorrect: 12

=== Classification Report ===
               precision    recall  f1-score   support

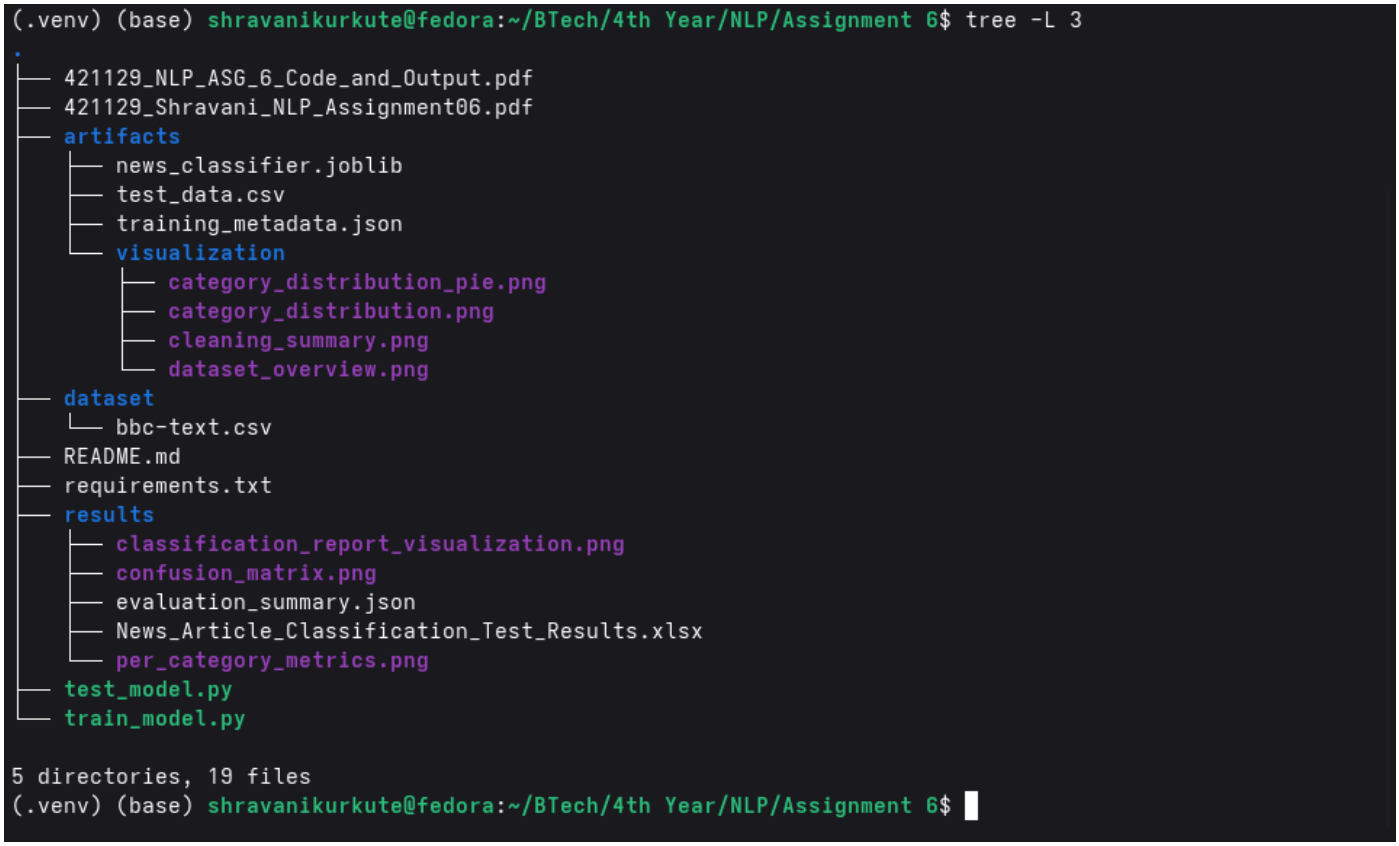
     business       0.95      0.97      0.96       100
entertainment       1.00      0.93      0.96        73
     politics       0.95      0.99      0.97        79
        sport       1.00      0.99      0.99        99
         tech       0.96      0.97      0.96        68

     accuracy                           0.97       419
    macro avg       0.97      0.97      0.97       419
 weighted avg       0.97      0.97      0.97       419

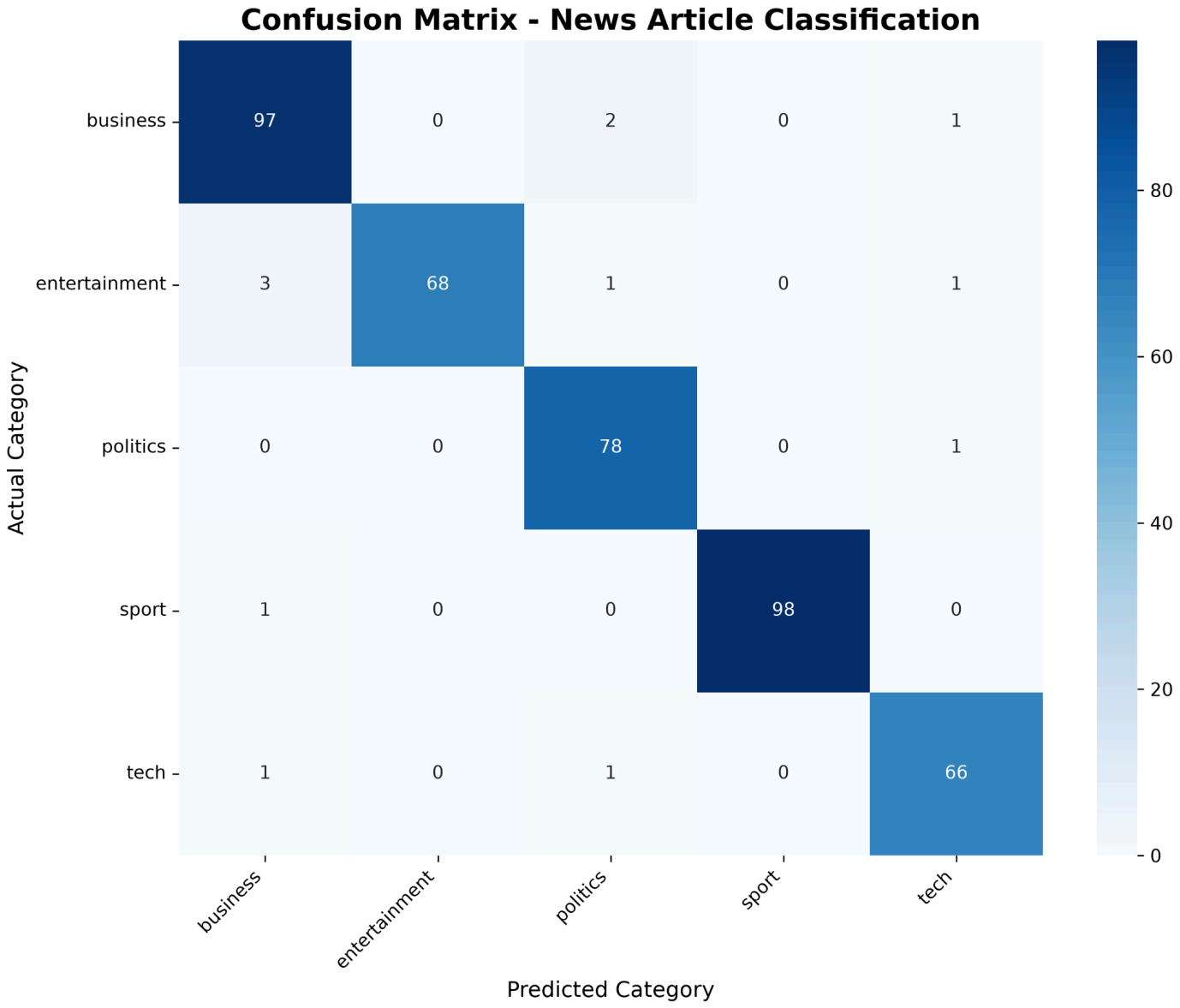

Generating visualizations...
Saved confusion matrix: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/results/confusion_matrix.png
Saved classification report visualization: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/results/classification_report_visualization.png
Saved per-category metrics chart: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/results/per_category_metrics.png

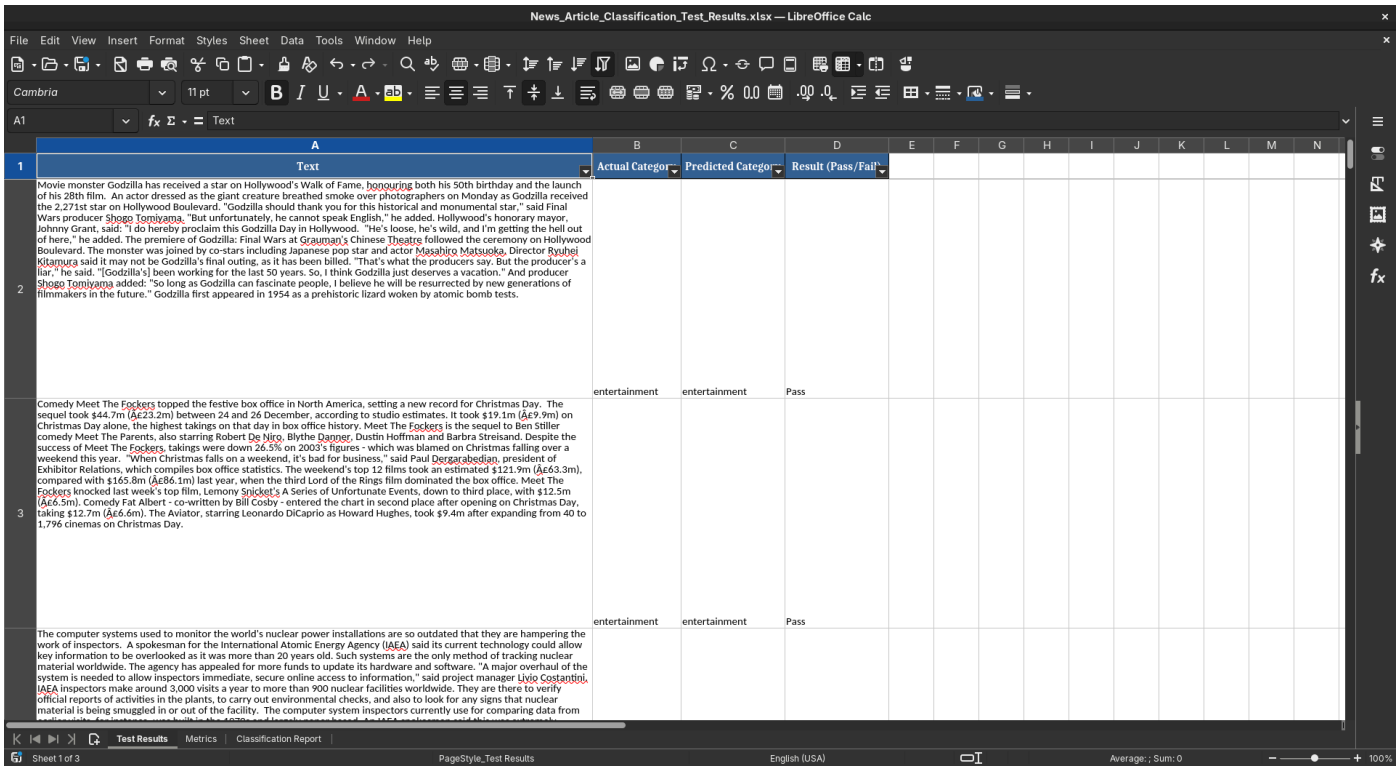
Creating Excel report...
Saved Excel report: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/results/News_Article_Classification_Test_Results.xlsx
Saved evaluation summary: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 6/results/evaluation_summary.json

Testing completed successfully!
(.venv) (base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 6$
```


(.venv) (base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 6$ tree -L 3
.
├── 421129_NLP_ASG_6_Code_and_Output.pdf
├── 421129_Shravani_NLP_Assignment06.pdf
├── artifacts
│   ├── news_classifier.joblib
│   ├── test_data.csv
│   ├── training_metadata.json
│   └── visualization
│   ├── category_distribution_pie.png
│   ├── category_distribution.png
│   ├── cleaning_summary.png
│   └── dataset_overview.png
├── dataset
│   └── bbc-text.csv
├── README.md
├── requirements.txt
├── results
│   ├── classification_report_visualization.png
│   ├── confusion_matrix.png
│   ├── evaluation_summary.json
│   ├── News_Article_Classification_Test_Results.xlsx
│   └── per_category_metrics.png
├── test_model.py
└── train_model.py
5 directories, 19 files
(.venv) (base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 6$

Output:


Confusion Matrix - News Article Classification
business
entertainment
politics
sport
tech
97
0
2
0
1
3
68
1
0
1
0
0
78
0
1
1
0
0
98
0
1
0
1
0
66
Actual Category
Predicted Category
80
60
40
20
0



	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Text	Actual Categor	Predicted Categor	Result (Pass/Fail)										
2	Movie monster Godzilla has received a star on Hollywood's Walk of Fame, honouring both his 50th birthday and the launch of his 28th film. An actor dressed as the giant creature breathed smoke over photographers on Monday as Godzilla received the 2,271st star on Hollywood Boulevard. "Godzilla should thank you for this historical and monumental star," said Final Wars producer Shogo Tomiyama. "But unfortunately, he cannot speak English," he added. Hollywood's honorary mayor, Johnny Grant, said: "I do hereby proclaim this Godzilla Day in Hollywood. "He's loose, he's wild, and I'm getting the hell out of here," he added. The premiere of Godzilla: Final Wars at Grauman's Chinese Theatre followed the ceremony on Hollywood Boulevard. The monster was joined by co-stars including Japanese pop star and actor Masahiro Matsuoka, Director Ryuhei Kitamura said it may not be Godzilla's final outing, as it has been billed. "That's what the producers say. But the producer's a liar," he said. "[Godzilla's] been working for the last 50 years. So, I think Godzilla just deserves a vacation." And producer Shogo Tomiyama added: "So long as Godzilla can fascinate people, I believe he will be resurrected by new generations of filmmakers in the future." Godzilla first appeared in 1954 as a prehistoric lizard woken by atomic bomb tests.	entertainment	entertainment	Pass										
3	Comedy Meet The Fockers topped the festive box office in North America, setting a new record for Christmas Day. The sequel took $44.7m (Â£23.2m) between 24 and 26 December, according to studio estimates. It took $19.1m (Â£9.9m) on Christmas Day alone, the highest takings on that day in box office history. Meet The Fockers is the sequel to Ben Stiller comedy Meet The Parents, also starring Robert De Niro, Blythe Danner, Dustin Hoffman and Barbra Streisand. Despite the success of Meet The Fockers, takings were down 26.5% on 2003's figures - which was blamed on Christmas falling over a weekend this year. "When Christmas falls on a weekend, it's bad for business," said Paul Dergarabedian, president of Exhibitor Relations, which compiles box office statistics. The weekend's top 12 films took an estimated $121.9m (Â£63.3m), compared with $165.8m (Â£86.1m) last year, when the third Lord of the Rings film dominated the box office. Meet The Fockers knocked last week's top film, Lemony Snicket's A Series of Unfortunate Events, down to third place, with $12.5m (Â£6.5m). Comedy Fat Albert - co-written by Bill Cosby - entered the chart in second place after opening on Christmas Day, taking $12.7m (Â£6.6m). The Aviator, starring Leonardo DiCaprio as Howard Hughes, took $9.4m after expanding from 40 to 1,796 cinemas on Christmas Day.	entertainment	entertainment	Pass										
	The computer systems used to monitor the world's nuclear power installations are so outdated that they are hampering the work of inspectors. A spokesman for the International Atomic Energy Agency (IAEA) said its current technology could allow key information to be overlooked as it was more than 20 years old. Such systems are the only method of tracking nuclear material worldwide. The agency has appealed for more funds to update its hardware and software. "A major overhaul of the system is needed to allow inspectors immediate, secure online access to information," said project manager Livio Costantini. IAEA inspectors make around 3,000 visits a year to more than 900 nuclear facilities worldwide. They are there to verify official reports of activities in the plants, to carry out environmental checks, and also to look for any signs that nuclear material is being smuggled in or out of the facility. The computer system inspectors currently use for comparing data from													